Chapter 7

7. Resampling methods.

7.1. Resampling.

7.2. Cross Validation.

7.3. Validation Set Approach.

7.4. Leave-One-Out Cross-Validation.

7.5. k-Fold Cross-Validation.

7.6. Bootstrapping.

Resampling

Basic principle: repeatedly draw samples from a training set and rerun a statistical procedure on the resampled data. This can e.g. be

- fitting a model;
- estimating parameters;
- classify data;
- etc.

The overall goal is to get an impression of how variable or how robust the targeted method is or to assess its performance.

We will discuss two main methods: cross-validiation and bootstrapping.

Cross-Validation

When fitting a statistical model we distinguish between a test error rate and a training error rate.

For the test error rate we validate the model using data that have not been used to fit the model. This is in contrast to the training error rate, where we estimate and test the model on the same data.

A major problem with the latter approach is overfitting. While a model may perfectly fit in-sample, the out-of-sample performance can be poor.

One way to avoid overfitting, is to include a penalty in the fitting procedure, where the complexity of the model is penalised. While the error decreases, the penalty term increases the more parameters we include in the model.

Cross-Validation

Another attempt is to use an out-of-sample evaluation of the model. This is what cross-validation is all about.

The goal of a CV can either be to assess the size of the test error

$$\text{“MSE}_{\text{test}}(\beta)\text{”}$$

or to find the model which minimizes the test error rate

$$\text{“argmin}_{\beta}\,\text{MSE}_{\text{test}}(\beta)\text{”}.$$

Here β represents the different competing models we investigate.

Cross-Validation

We compare a few methods. For illustrative purpose we use the data `Auto` from the `ISLR` package.

```
> library(ISLR))
# The data set Auto contains 392 data points, each with 9 components
> head(Auto)
  mpg cylinders displacement horsepower weight acceleration year origin name
1  18         8          307        130   3504         12.0   70      1
2  15         8          350        165   3693         11.5   70      1
3  18         8          318        150   3436         11.0   70      1
4  16         8          304        150   3433         12.0   70      1
5  17         8          302        140   3449         10.5   70      1
6  15         8          429        198   4341         10.0   70      1
```

Validation Set Approach

- Split the sample randomly in a training set and a test set (hold-out set, validation set).
- Fit the model on the training set.
- Predict responses in the validation set.
- Check the quality of predictions (e.g. using MSE, or misclassification error rate, etc.)

Validation Set Approach

Problems

- Depends on the data split and can then lead to quite different results.
- We choose only a subset for model fitting. E.g. if we split the data 50:50 we loose half of our observations in the estimation, which will result in less accurate models.

Validation Set Approach

```
# randomly choose 50% of the data and use them as training set
> train <- sample(392,196)

# fit a linar model to describe miles per gallon by weight and horsepower
> fit <- lm(mpg~weight+horsepower, data=Auto,subset=train)

# check performance on validation set
> attach(Auto)
> mean((mpg-predict(fit,Auto))[-train]^2)

18.36
```

Validation Set Approach

Does the performance improve if we take into account powers of covariates?

We use powers up to order 5 for each covariate. This gives altogether 25 combinations and for each we compute the MSE.

```
> MSE <- data.frame(matrix(0,ncol=5,nrow=5))
> colnames(MSE) <- c("hp1","hp2","hp3","hp4","hp5")
> rownames(MSE) <- c("we1","we2","we3","we4","we5")
> for(k in 1:5){
+   for(j in 1:5){
+     fitkj <- lm(mpg~poly(horsepower,j)+poly(weight,k),
+                 data=Auto,subset=train)
+     MSE[k,j] <- mean((mpg-predict(fitkj,Auto))[-train]^2)
+   }
+ }
>
> xtable(MSE)
```

Validation Set Approach

	hp1	hp2	hp3	hp4	hp5
we1	18.79	16.25	15.82	16.62	16.07
we2	16.73	15.86	15.89	16.45	16.06
we3	16.73	15.85	15.88	16.66	16.03
we4	16.53	15.60	15.58	16.37	15.81
we5	16.59	15.71	15.73	16.50	15.94

Validation Set Approach

Running the same procedure again, we get different MSEs and a different best performing model.

	hp1	hp2	hp3	hp4	hp5
we1	17.45	15.00	14.53	14.60	14.10
we2	15.20	14.36	14.26	14.37	13.97
we3	15.22	14.35	14.20	14.30	13.81
we4	15.12	14.37	14.27	14.44	14.01
we5	15.11	14.30	14.21	14.38	14.02

Leave-One-Out Cross-Validation

- A single observation is used for the validation, all other data are used as training set.
- The procedure is repeated n times with holding out each time a different observation for the validation.
- E.g. if the error is measured by the MSE, then MSE_i is $(y_i - \hat{y}_i)^2$, where $\hat{y}_i$ is computed from leaving out y_i. Then we determine

$$\text{CV}_{(n)} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

- If $\hat{y}_i = \hat{y}_i(\beta)$ then for model selection we choose β such that $\text{CV}_{(n)}(\beta)$ is minimized.

Leave-One-Out Cross-Validation

Advantages:

- We use almost the entire sample for estimation and hence we will not overestimate the test error.
- Will always yield the same result, because it doesn't depend on random data splits.

The downside of the method is that it might be numerically rather costly.

Leave-One-Out Cross-Validation

With data:

```
# instead of lm we need to use the function glm
# the file Auto is still attached
> fit=glm(mpg~weight+horsepower)

# we need library "boot"
> library("boot")
> CVn=cv.glm(Auto,fit)
# this command gives our LOOCV error estimate
> CVn$delta[1]
```

The result (here 18.11) is no longer random!

Leave-One-Out Cross-Validation

We do again a table with different powers of the covariates:

	hp1	hp2	hp3	hp4	hp5
we1	18.11	16.19	15.96	16.09	15.83
we2	16.26	15.71	15.76	15.90	15.74
we3	16.32	15.75	15.78	15.95	15.72
we4	16.22	15.63	15.66	15.81	15.63
we5	16.27	15.67	15.71	15.87	15.70

k-Fold Cross-Validation

- Randomly split the data into k groups, of approximately equal size.
- Instead of leaving out a single observation for the validation, we leave out an entire group.
- Define $\text{MSE}_i = \sum_{\ell \in \text{group } i} (y_\ell - \hat{y}_\ell)^2$, where $\hat{y}_\ell$ is computed from leaving out the data in group i. Then we determine

$$\text{CV}_{(k)} = \sum_{i=1}^{k} \text{MSE}_i.$$

- If $k = n$, then k-fold CV coincides with LOOCV.

k-Fold Cross-Validation

Advantages:

- Is similar in its spirit as LOOCV.
- Is numerically less intense than LOOCV.
- Out-of-sample error variance estimation with k-Fold CV tends to have a lower variance. (See next slides).

k-Fold Cross-Validation

- The validation set approach will typically overestimate the test error, because we use little data for estimation.
- With this logic, LOOCV will give the smallest bias:

$$\text{bias}(\text{MSE}_{\text{test}}^{\text{LOOCV}}) < \text{bias}(\text{MSE}_{\text{test}}^{\text{k-CV}}) < \text{bias}(\text{MSE}_{\text{test}}^{\text{Validation}}).$$

- However, (perhaps surprisingly) it turns out that LOOCV has a larger variance, then k-fold CV.
- The reason for this is that all models in LOOCV are highly correlated (only one sampling point is changed).

k-Fold Cross-Validation

We run again a loop and compute $CV_{(k)}$ for different models.

```
> CVkMSE <- data.frame(matrix(0,ncol=5,nrow=5))
> colnames(CVkMSE) <- c("hp1","hp2","hp3","hp4","hp5")
> rownames(CVkMSE) <- c("we1","we2","we3","we4","we5")
> for(k in 1:5){
+   for(j in 1:5){
+     fitkj <- glm(mpg~poly(weight,k)+poly(horsepower,j))
+     CVkMSE[k,j] <- cv.glm(Auto,fitkj,K=K)$delta[1]
+   }
+ }
>
> xtable(CVkMSE)
```

k-Fold Cross-Validation

	hp1	hp2	hp3	hp4	hp5
we1	17.97	16.26	15.96	16.17	15.92
we2	16.35	15.74	15.56	15.69	15.90
we3	16.25	15.92	15.77	16.09	15.94
we4	16.31	15.60	15.61	15.78	15.90
we5	16.37	15.89	15.81	15.72	15.80

Bootstrap

Suppose we have a random sample $X = \{X_1, \ldots, X_n\}$ from a population F.

For inferential problems we consider some statistics $T = T(X)$. E.g. if we want to estimate the mean ($T = \bar{X}$) or some model parameters ($T = \hat{\theta}$).

We then want to know (the approximate) distribution of the statistic, e.g. for constructing confidence intervals.

Bootstrap

Example 1: Suppose we have a random sample from a $N(\mu, \sigma^2)$ population. The goal is to estimate μ.

We set $T = \bar{X}$ and know that $\bar{X} \sim N(\mu, \sigma^2/n)$. Using Fisher's theorem we know that

$$\mu \in \left[\bar{X} - \frac{S}{\sqrt{n}} t_{n-1,\alpha/2}, \bar{X} + \frac{S}{\sqrt{n}} t_{n-1,\alpha/2}\right]$$

with probability $1 - \alpha$.

If n is large enough, this remains approximately true even if the normality assumption doesn't hold.

Bootstrap

In general it is difficult to obtain the distribution of a statistic.

Especially the finite sample distributions are hard to come by.

A (theoretical) way to estimate the distribution for arbitrary T is to rerun the experiment X several times (say B times) independently and then create a random sample $T_1, \ldots, T_B$.

All T_i's have the same distribution as $T \Longrightarrow$ a histogram of the T_i's estimates the density of T.

Bootstrap

With this approach we need not n but $B \times n$ observations.

$$X_1, \ldots, X_n \quad \Longrightarrow \quad \begin{array}{ll} X_1, \ldots & \ldots, X_n \\ X_{n+1}, \ldots & \ldots, X_{2n} \\ \ldots & \\ \ldots & \\ X_{(B-1)n+1}, \ldots & \ldots, X_{Bn} \end{array}$$

Practically infeasible!

Bootstrap

The idea of the bootstrap is to ,,recycle“ the sample, i.e. to resample.

$$X_1, \ldots, X_n \quad \Longrightarrow \quad \begin{array}{ll} X_1^*, \ldots & \ldots, X_n^* \\ X_{n+1}^*, \ldots & \ldots, X_{2n}^* \\ \ldots & \\ \ldots & \\ X_{(B-1)n+1}^*, \ldots & \ldots, X_{Bn}^* \end{array}$$

The X_i^* are obtained from X by drawing with replacement, such that $P(X_i^* = X_k) = \frac{1}{n}$.

Note: the X_i^*'s form an iid sample from $\hat{F}$.

Bootstrap

From the B bootstrap samples we obtain B independent bootstrap replicates of T:

$$
\begin{array}{lll}
X_1^*, \ldots & \ldots, X_n^* & \Longrightarrow \quad T_1^* \\
X_{n+1}^*, \ldots & \ldots, X_{2n}^* & \Longrightarrow \quad T_2^* \\
\ldots & & \\
\ldots & & \\
X_{(B-1)n+1}^*, \ldots & \ldots, X_{Bn}^* & \Longrightarrow \quad T_B^*
\end{array}
$$

When F_n and F are close, then we expect that

$$P(T \leq t) \approx P(T^* \leq t|X) = \lim_{B \to \infty} \frac{1}{B} \sum_{i=1}^{B} I\{T_i^* \leq t\}.$$

The latter equality is just the law of large numbers.

Bootstrap

For example, if we want to estimate $\mathrm{sd}(T)$, then we may use the bootstrap estimator of $\mathrm{sd}(T)$, defined by

$$\hat{\mathrm{sd}}_{\mathrm{Boot}}(T) = \left(\frac{1}{B-1} \sum_{b=1}^{B} (T_b^* - \bar{T}^*)^2 \right)^{1/2}.$$

Bootstrap

We have explained the simplest setting for the bootstrap. For example, if the data are not iid, then there are methods to adapt the bootstrap (e.g. sampling from residuals $\Longrightarrow$ sieve bootstap). We do not go into further details.

Let us consider two numerical illustrations.

Bootstrap

Example 1: Suppose we have a random sample $X_1, \ldots, X_{30}$ from a $N(\mu, \sigma^2)$ population.

We do a small simulation study where we choose $\mu = 0$ and $\sigma^2 = 1$. From our theory we know that

$$S^2 \sim \frac{\sigma^2}{29}\chi^2_{29}.$$

Alternatively we can directly sample from the true data and derive the distribution of S^2 via Monte-Carlo simulation.

Since we have simulated data we know the exact distribution of $S = \text{sd}(X)$ (because we know $\sigma^2 = 1$).

Let us compare the bootstrap distribution of S with the true distribution of S.

Bootstrap

```
> x <- rnorm(n=30,0,1)

# bootstapping hat.S
> hat.S.star <- c()
> for(i in 1:10000){
+   x.star <- sample(x,size=30,replace=TRUE)
+   hat.S.star <- c(sd(x.star)*sqrt(30/29),hat.S.star)
+ }

# generating hat.S by Monte Carlo
> hat.S <- c()
> for(i in 1:10000){
+   x.i <- rnorm(n=30,0,1)
+   hat.S <- c(sd(x.i)*sqrt(30/29),hat.S)
+ }
```

Bootstrap

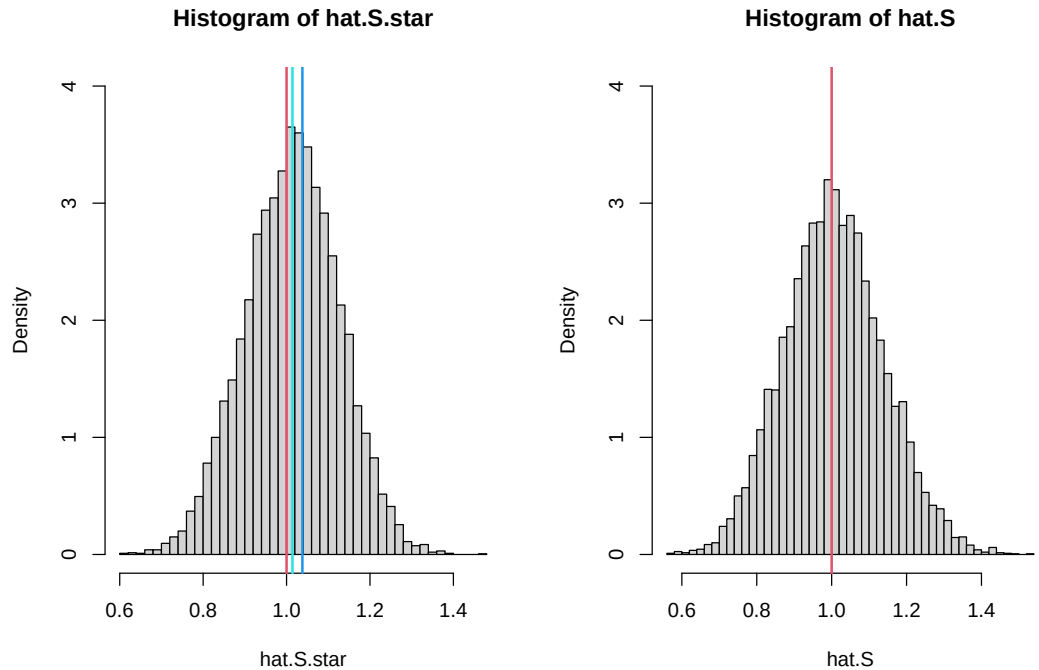


Figure 1: Blue line equals $\widehat{\text{sd}}(X)$ and red line $\text{sd}(X)$. Cyan line is average$(\widehat{\text{sd}}(X_i^*))$. Left is the histogram of 10k bootstrap samples, right the histogram of 10k MC samples.

Bootstrap

We can also estimate the standard deviation of S (standard deviation of an empirical standard deviation ...). The bootstap estimator of $\mathrm{sd}(S)$ is

$$\sqrt{\frac{1}{B-1}\sum_{i=1}^{B}(S_i^* - \overline{S^*})^2} \approx 0.11$$

compared to 0.13 which is the true value of $\mathrm{sd}(S)$.

Bootstrap

Example 2: We do a PCA and estimate the amount of variance explained by the first principal component.

$$\frac{\lambda_1}{\lambda_1 + \cdots + \lambda_p}.$$

Obvious estimator is

$$T(X) = \frac{\hat{\lambda}_1}{\hat{\lambda}_1 + \cdots + \hat{\lambda}_p},$$

where $\hat{\lambda}_i$ are the eigenvalues of $\frac{1}{n}\sum_{k=1}^{n}(X_k - \bar{X})(X_k - \bar{X})'$.

How good is this estimator? We want to determine $\mathrm{sd}(T(X))$ ...